



CyberRad

Manual técnico completo, guía de instalación, operación, mantenimiento y seguridad

Documento elaborado sobre el paquete adjunto cyberrad_v5_1_16_hotfix34_sector_loader_feedback.zip

Edición profesional presentada por Timblau Industries

Este manual está pensado para que un tercero pueda entender cómo está construido CyberRad, instalarlo desde cero, ponerlo en marcha, mantenerlo de forma estable y defender técnicamente su comportamiento cuando está bien desplegado.

Alcance analizado: carpeta extraída cyberrad_hotfix25, aplicación FastAPI, base de datos SQLite, frontend web estático, utilidades de despliegue y pruebas smoke incluidas en el adjunto.

Abril de 2026

Nota de alcance y control de versión

El paquete adjunto presenta varias referencias de versión mezcladas dentro del propio proyecto. La carpeta extraída se llama `cyberrad_hotfix25`; el HTML muestra CyberRad V5.1.13 Hotfix 34 en el título y V5.1.10 en el encabezado visible; el README habla de V4.5.4; y el nombre del ZIP menciona V5.1.16 Hotfix 34. Por ello, este manual documenta el comportamiento real del código entregado, no una etiqueta comercial única.

Esta aclaración es importante para soporte y mantenimiento: cualquier inventario interno debería fijar un identificador único de release antes de distribuir el proyecto a terceros. Mientras no se normalice esa nomenclatura, la referencia más fiable es la suma carpeta extraída + hash del ZIP + fecha de entrega.

Índice de contenidos

Las entradas del índice de esta versión PDF son navegables y llevan directamente a la sección correspondiente.

1. Introducción general.....	4
2. Requisitos técnicos mínimos y recomendados	4
3. Arquitectura general del proyecto	5
4. Flujo operativo completo: de la fuente a la interfaz	7
5. Explicación archivo por archivo.....	7
6. Funcionamiento interno detallado	18
6.1 Carga de fuentes	18
6.2 Recogida de noticias.....	19
6.3 Filtrado y clasificación	19
6.4 Gestión de imágenes	19
6.5 Decisión de ubicación por país	19
6.6 Presentación web y actualización del sistema	20
7. Seguridad del proyecto	20
7.1 Buenas prácticas básicas de seguridad al instalarlo	21
7.2 Riesgos realistas de despliegue y mitigación por escenario	21
8. Instalación completa desde cero.....	22
8.1 Instalación recomendada con Docker en Kali o Linux similar	22
8.2 Instalación nativa con Python	22
8.3 Dependencias, estructura y variables relevantes.....	23
8.4 Instalación de Docker en Ubuntu y despliegue con runner	23
8.5 Cambio de puerto y exposición controlada.....	23
9. Puesta en marcha y uso diario	23
9.1 Cómo comprobar que funciona bien.....	24
9.2 Qué significan las secciones y controles principales	24
9.3 Uso recomendado del operador	25
9.4 Qué esperar en el primer arranque	25
10. Mantenimiento, validación y solución de problemas	25
10.1 Errores comunes y posibles causas	25
10.2 Qué revisar si fallan noticias, imágenes, mapa o filtros.....	26
10.3 Validaciones útiles de funcionamiento	26
10.4 Copia de seguridad y restauración	26
11. Buenas prácticas operativas.....	27
11.1 Cómo añadir o quitar fuentes correctamente	27
11.2 Cómo actualizar sin romper el sistema	27
12. Conclusión.....	27
Anexo A. Resumen de la base de datos.....	28
Anexo B. Rutas API principales.....	28
Anexo C. Resumen rápido de comandos.....	29

1. Introducción general

CyberRad es una aplicación web de vigilancia y priorización de noticias e incidentes de ciberseguridad. Su función principal es recoger entradas desde feeds RSS, Atom y JSON, enriquecerlas con metadatos útiles, clasificarlas con lógica local y mostrarlas en una interfaz operativa con filtros, estados, mapa por países y exportación documental.

En términos prácticos, CyberRad resuelve un problema muy habitual en monitorización: el exceso de fuentes, el ruido de titulares ambiguos y la dificultad de convertir un gran volumen de noticias en un panel utilizable. En lugar de limitarse a listar titulares, el sistema incorpora reglas de relevancia, extracción de entidades, deduplicación, agrupación temática y segmentación por sectores, incluida una vista específica para marcas y retail.

La información que analiza o muestra incluye título, resumen, contenido textual del feed, enlace original, fecha de publicación, imagen relacionada, vídeo relacionado cuando existe, tags detectadas, puntuación de relevancia, sector asignado, agrupación por duplicados, clúster temático y país inferido para representación cartográfica.

La idea general del sistema es sencilla pero potente: mantener una base local de artículos procedentes de fuentes activas, recalcular su relevancia según configuración interna y feedback del operador, y presentarlos en una interfaz que permita decidir rápido qué es relevante, qué debe quedar pendiente y qué se descarta.

CyberRad incorpora técnicas de inteligencia artificial aplicadas al análisis, filtrado, contextualización y priorización de información de ciberseguridad. Sin depender de servicios de IA externos, el sistema emplea análisis semántico, extracción de entidades, correlación temática y aprendizaje asistido por feedback para refinar de forma progresiva la relevancia del contenido mostrado. Este enfoque posiciona a CyberRad como una solución avanzada de análisis automatizado y apoyo a la decisión, concebida para aportar criterio técnico, consistencia operativa y control local sobre los datos procesados.

2. Requisitos técnicos mínimos y recomendados

Los siguientes requisitos se proponen como guía realista de operación basándose en el código entregado, los perfiles de rendimiento definidos y el trabajo que realizan FastAPI, SQLite, scikit-learn, Pillow y la recogida concurrente de feeds. No aparecen como tabla oficial en el código, pero sí son una estimación técnica razonable para dejar el proyecto operativo.

CPU	Mínimo práctico: 4 hilos lógicos. Recomendado: 6 a 8 hilos para refrescos más fluidos y re-score rápido.
Memoria RAM	Mínimo: 8 GB, funcionando justo y con tiempos de carga más altos. Recomendado: 16 GB para una experiencia estable y margen suficiente.
Disco	Mínimo: 5 GB libres. Recomendado: 15 GB o más para base de datos, exportaciones, logs, imágenes temporales y crecimiento del histórico.
Sistema operativo	Linux recomendado. El paquete incluye rutas y scripts orientados a Ubuntu, Linux Mint y Kali. Para Windows, sería preferible WSL o contenedor.
Python	Python 3.12 si se ejecuta en modo nativo con el requirements.txt incluido.
Contenedores	Docker Engine + Docker Compose Plugin recomendados para despliegues repetibles.
Red	Salida HTTP/HTTPS para leer feeds, abrir artículos y descargar imágenes. No requiere una base de datos de red externa.
Puertos	Por defecto 3000 o 3001 según compose. Se recomienda exponer solo por localhost o detrás de proxy/VPN.

Interpretación práctica: con 8 GB de RAM CyberRad puede funcionar, especialmente en perfil low_resource, pero irá más apretado cuando haya muchas noticias, reprocesados o imágenes. Con 16 GB de RAM el sistema gana margen para el refresco de feeds, la deduplicación y la interacción más cómoda con el panel.

Perfiles disponibles en el código: low_resource, balanced y full. Los tres mantienen el conjunto de feeds permitidos, pero cambian la concurrencia, el máximo de artículos por feed, la ventana temporal y la compresión de imágenes. Eso permite adaptar el consumo del sistema sin cambiar la arquitectura.

3. Arquitectura general del proyecto

CyberRad está organizado como una aplicación web monolítica ligera. No hay separación en microservicios ni un frontend desacoplado de SPA compilada. El backend FastAPI sirve tanto la API como la plantilla HTML y los recursos estáticos, y la persistencia se resuelve localmente con SQLite. Esa simplicidad reduce complejidad de operación y facilita desplegar el proyecto en una única máquina.

La arquitectura funcional se compone de varias piezas coordinadas. El backend en `app/main.py` actúa como núcleo de ejecución, expone las rutas HTTP, lanza el refresco de feeds al arrancar y programa los re-score cuando cambia la configuración o cuando el operador reclasifica noticias. El módulo `app/db.py` gestiona la base SQLite, los feeds configurados, los ajustes y la escritura/lectura de artículos. El módulo `app/rss.py` hace la ingesta real desde feeds, limpia el contenido, intenta localizar imágenes y decide una primera clasificación léxica.

Sobre esa base, `app/intelligence.py` enriquece cada artículo con entidades, similitud semántica, aprendizaje a partir del feedback y clústeres de duplicados o campañas. `app/incident_map.py` y `app/brands_map.py` transforman artículos ya procesados en datos geográficos utilizables para el mapa de General y el de Marcas. `app/exporter.py` genera exportaciones DOCX. El frontend, compuesto por `index.html`, `app.js` y `styles.css`, consume la API, muestra los datos, aplica cachés de interfaz y ofrece interacción con filtros, mapa y artículos.

La relación entre backend, frontend, base de datos, fuentes, procesamiento, filtros, mapa, imágenes y clasificación es directa: las fuentes alimentan la base; el backend clasifica y reordena; la API entrega vistas ya preparadas; el frontend presenta esas vistas y permite al usuario modificarlas; los cambios del usuario retroalimentan la puntuación y terminan persistiendo en SQLite.

Backend HTTP	FastAPI sirve HTML, archivos estáticos y API REST para estado, feeds, artículos, cambios de estado, mapas e imagen-proxy.
Persistencia	SQLite en <code>data/cyberad_v4.db</code> almacena feeds, ajustes, artículos y eventos de feedback.
Ingesta	requests descarga feeds XML/JSON y, cuando hace falta, abre la noticia original para enriquecer imágenes o vídeo.
Clasificación	Heurísticas léxicas + TF-IDF/coseno + aprendizaje por feedback + deduplicación temática.
Mapas	Inferencia de país, agrupación por año, sector y tipo de incidente, y proyección sobre mapa base SVG/PNG.
Frontend	Plantilla HTML y JavaScript sin build complejo. El navegador consulta <code>/api/*</code> y renderiza panel, tarjetas y mapa.
Exportación	Exportación a Word de noticias incluidas y PDF del mapa de marcas.

4. Flujo operativo completo: de la fuente a la interfaz

1. Al arrancar la aplicación, FastAPI ejecuta `init_db()`, `seed_default_feeds()` y crea tareas de refresco y rescoring en background dentro del ciclo de vida de la app.
2. La base SQLite se asegura de tener tablas, columnas y ajustes mínimos; además sincroniza los feeds permitidos según el perfil runtime activo.
3. El refresco inicial llama a `refresh_all_feeds()`. Los feeds prioritarios de Marcas pueden procesarse primero y después se revisa el resto del conjunto activo.
4. Cada feed se descarga con requests, se interpreta como XML RSS/Atom o JSON Feed, y de cada entrada se extraen título, enlace, resumen, contenido, fecha, imagen y vídeo si existen.
5. `rss.py` limpia HTML, normaliza texto, detecta términos ciber relevantes y decide una primera puntuación y estado: `included`, `pending` o `excluded`.
6. Los artículos se insertan o actualizan por `unique_key` en SQLite. Si un artículo ya había sido reclasificado manualmente, el sistema respeta su status manual y solo refresca metadatos técnicos.
7. Después de la ingesta, `rescore_articles()` vuelve a analizar toda la base con `intelligence.py`: extracción de entidades, similitud semántica, aprendizaje por feedback, sectorización y clustering.
8. La API entrega artículos visibles, estadísticas, feeds, mapas y detalle por país. Para el listado principal se mezcla el orden de fuentes para evitar que un agregador domine toda la pantalla.
9. El frontend consulta `/api/state`, `/api/articles` y `/api/incidents-map`, pinta tarjetas con imagen, texto y botones de estado, y mantiene cachés de UI para que la navegación sea más fluida.
10. Cuando el usuario cambia ajustes, añade feeds o reclasifica una noticia, el backend persiste el cambio y programa un nuevo rescoring para mantener la coherencia del panel.

5. Explicación archivo por archivo

En esta sección se describen los archivos que realmente tienen función operativa o de soporte relevante. No se incluyen los `.pyc` de `__pycache__` porque son artefactos de ejecución, no piezas del diseño. Tampoco se trata como “núcleo” cualquier copia `.bak`, aunque sí se documenta por su impacto de mantenimiento.

README.md

Ubicación	raíz del proyecto
Para qué sirve	Documento de contexto rápido para despliegue en Icarus/GitLab Runner y pruebas en Kali.
Depende de	No depende del runtime; referencia Docker, Docker Compose, uvicorn y la estructura general del repo.
Parte del flujo que afecta	Afecta a instalación y operación humana, no al flujo de ejecución del software.

Resume cómo levantar la aplicación tanto sin Docker como con Docker. También explica la idea de publicar por 127.0.0.1 y mover el servicio a 3001 en despliegues donde el 3000 ya esté ocupado. Hay una referencia a `.gitlab-ci.yml` que no aparece dentro del ZIP entregado, por lo que debe considerarse una pista de despliegue, no un artefacto realmente incluido en este adjunto.

requirements.txt

Ubicación	raíz del proyecto
-----------	-------------------

Para qué sirve	Lista cerrada de dependencias Python para instalación determinista.
Depende de	Se usa desde pip y desde Dockerfile.
Parte del flujo que afecta	Afecta al arranque completo porque define FastAPI, uvicorn, BeautifulSoup, python-docx, scikit-learn, reportlab, pycountry, countryinfo y Pillow.

Es la base del runtime Python nativo y del build del contenedor. Su presencia con versiones fijadas favorece reproducibilidad. Para mantenimiento, cualquier actualización debería probarse junto con smoke tests y una revisión de compatibilidad del módulo de clasificación.

Dockerfile

Ubicación	raíz del proyecto
Para qué sirve	Construcción del contenedor principal de CyberRad.
Depende de	requirements.txt y todo el árbol del proyecto.
Parte del flujo que afecta	Afecta al despliegue y a la postura de seguridad del contenedor.

Parte de python:3.12-slim, instala build-essential y curl, copia el proyecto y crea un usuario no root llamado cyberrad. Expone el puerto 3000 y define un HEALTHCHECK contra /api/state. Es importante porque reduce privilegios de ejecución, evita correr como root y deja una base bastante limpia para despliegues en host Linux.

docker-compose.yml

Ubicación	raíz del proyecto
Para qué sirve	Compose principal orientado a despliegue más seguro o integrado en entornos tipo Icarus.
Depende de	Dockerfile y volumen ./data.
Parte del flujo que afecta	Afecta a cómo se publica el servicio, persiste SQLite y se aplican restricciones del contenedor.

Por defecto publica 127.0.0.1:3001 -> 3000, usa el UID/GID del host, activa no-new-privileges, hace cap_drop ALL, monta /tmp como tmpfs y añade un healthcheck. Es un archivo especialmente relevante para justificar que el proyecto no intenta exponerse a toda la red por defecto.

docker-compose.kali.yml

Ubicación	raíz del proyecto
Para qué sirve	Compose simplificado para pruebas rápidas en Kali Linux.
Depende de	Dockerfile y volumen ./data.
Parte del flujo que afecta	Afecta a despliegue local en laboratorio.

Publica 127.0.0.1:3000 -> 3000 y mantiene varias medidas de endurecimiento del compose principal. Es la opción más cómoda para levantar la aplicación localmente sin pelearse con puertos alternativos.

app/main.py

Ubicación	app/main.py
Para qué sirve	Núcleo de la aplicación web y punto de entrada de FastAPI.
Depende de	app/db.py, app/rss.py, app/incident_map.py, app/brands_map.py y app/exporter.py.
Parte del flujo que afecta	Afecta a todo el flujo: arranque, API, refresco, rescoring, cachés, imagen-proxy, mapas y exportación.

Contiene la definición de la aplicación, las tareas de startup, el bucle de auto-refresco, el programador de rescoring y todas las rutas HTTP relevantes. También implementa cachés temporales para artículos, mapas, estado e imágenes, y una mezcla equilibrada de fuentes para que la vista general no quede monopolizada por agregadores. Es el archivo que más claramente expresa la arquitectura real del proyecto.

app/db.py

Ubicación	app/db.py
Para qué sirve	Capa de persistencia y configuración.
Depende de	SQLite y, para rescoring, app/intelligence.py.
Parte del flujo que afecta	Afecta a inicialización, schema, feeds, ajustes, artículos, feedback y estadísticas.

Define el path de la base, crea las tablas feeds, articles, settings y feedback_events, mantiene presets de runtime y sincroniza el conjunto de feeds permitidos. También protege la semántica de status manual frente a actualizaciones automáticas. Dentro del archivo hay trazas de evolución y duplicación de definiciones propias de varios hotfix; funcionalmente sigue siendo la pieza central de la persistencia.

app/rss.py

Ubicación	app/rss.py
Para qué sirve	Ingesta, limpieza, parsing de feeds y primera clasificación.
Depende de	requests, BeautifulSoup y utilidades internas del propio módulo.
Parte del flujo que afecta	Afecta a la captura de noticias, detección de imágenes, tags, score inicial y clasificación por sector.

Es uno de los módulos más importantes del proyecto. Interpreta feeds XML y JSON, intenta localizar imágenes y vídeos, abre la noticia original cuando el feed no trae media suficiente y calcula una puntuación contra el prompt del operador. También contiene la lógica inicial para el sector marcas_retail, reglas de ambigüedad, bonus por fuente especializada y ventanas temporales.

app/intelligence.py

Ubicación	app/intelligence.py
Para qué sirve	Reanálisis avanzado de artículos ya almacenados.
Depende de	app/entities.py, app/semantic.py, app/feedback.py, app/clustering.py y app/rss.py.
Parte del flujo que afecta	Afecta al rescoring global, a las etiquetas finales, al sector visible y a la deduplicación.

Toma los artículos de la base y les añade entidades, scoring semántico, aprendizaje por feedback, categorías sectoriales y metadatos de clustering. Su output acaba escribiéndose de nuevo en SQLite y es lo que permite que la interfaz muestre un score más rico que el mero filtrado léxico inicial.

app/entities.py

Ubicación	app/entities.py
Para qué sirve	Extracción ligera de entidades técnicas y de contexto.
Depende de	Solo expresiones regulares y estructuras internas del módulo.
Parte del flujo que afecta	Afecta al texto enriquecido, a la similitud semántica, a los clústeres y al etiquetado técnico.

Localiza CVE, vendors, productos, familias de malware, actores y tácticas con patrones controlados. No usa NLP pesado ni modelos externos; trabaja con listas y expresiones regulares. Es una decisión pragmática: menos sofisticación lingüística, pero mayor control y menor dependencia.

app/semantic.py

Ubicación	app/semantic.py
Para qué sirve	Cálculo de similitud semántica local.
Depende de	app/entities.py y app/rss.py, además de scikit-learn.
Parte del flujo que afecta	Afecta a la puntuación híbrida de cada artículo.

Expande el prompt del operador con variantes y tags del léxico, construye un documento enriquecido del artículo y calcula similitud TF-IDF tanto a nivel palabra como a nivel char_wb. No hay embeddings remotos ni vector database; el comportamiento es reproducible y local.

app/feedback.py

Ubicación	app/feedback.py
Para qué sirve	Aprendizaje ligero a partir de reclasificaciones manuales.
Depende de	app/entities.py y collections.Counter.
Parte del flujo que afecta	Afecta a la parte de feedback de la puntuación híbrida.

Construye un perfil simple a partir de noticias incluidas y excluidas manualmente, y calcula sesgos por tokens, tags y dominios mediante log-odds acotados. No sustituye la lógica principal, pero ayuda a orientar el sistema hacia el criterio real del operador.

app/clustering.py

Ubicación	app/clustering.py
Para qué sirve	Agrupación de duplicados y campañas relacionadas.
Depende de	app/entities.py, app/rss.py y scikit-learn.
Parte del flujo que afecta	Afecta a duplicate_group, duplicate_rank y campaign_cluster.

Usa TF-IDF y cosine similarity junto con señales de CVE, vendors, productos y proximidad temporal para unir noticias que hablan del mismo incidente o de una misma campaña. Esto permite ocultar duplicados y dar contexto temático sin perder la noticia principal.

app/incident_map.py

Ubicación	app/incident_map.py
Para qué sirve	Motor del mapa público y del filtrado sectorial visible.
Depende de	app/brands_map.py y app/rss.py.
Parte del flujo que afecta	Afecta al mapa General, al mapa de Marcas dentro de la API unificada y al drill-down por país.

Analiza la relevancia real de un artículo para General o Marcas, infiere país, etiqueta dominante y categorías de contenido, agrupa incidentes por país y año y produce el JSON que consume el frontend. Es clave porque evita que el mapa se limite a “cualquier cosa con un país” y exige contexto ciber o digital real.

app/brands_map.py

Ubicación	app/brands_map.py
Para qué sirve	Soporte cartográfico específico para Marcas y exportación PDF de ese mapa.
Depende de	app/rss.py, reportlab, pycountry y countryinfo.
Parte del flujo que afecta	Afecta a la inferencia país-marca, a la proyección cartográfica y al PDF del mapa.

Incluye alias de países, marcas asociadas a países, palabras clave de tipos de ataque y coordenadas/proyección sobre el mapa base. Además genera el PDF del mapa anual de marcas con ReportLab. Es un módulo importante porque concentra la lógica geográfica especializada del proyecto.

app/exporter.py

Ubicación	app/exporter.py
Para qué sirve	Exportación documental de noticias a Word.
Depende de	python-docx y requests.
Parte del flujo que afecta	Afecta a la salida documental cuando el usuario exporta noticias incluidas.

Genera un DOCX con título, fuente, fecha, estado, scores, clusters, texto e imagen. Es un exportador sencillo pero funcional. No altera el procesamiento principal; trabaja sobre artículos ya clasificados y persistidos.

app/templates/index.html

Ubicación	app/templates/index.html
Para qué sirve	Plantilla principal de la interfaz web.
Depende de	Jinja2, estilos CSS y JavaScript del frontend.
Parte del flujo que afecta	Afecta a la presentación del panel y a la estructura visible de controles.

Define la topbar, el panel lateral de estado/configuración/feeds, los filtros de status y sector, el bloque del mapa anual y el contenedor de artículos. También deja visibles las opciones de runtime profile, relevancia, auto-refresh, ocultación de duplicados y exportación Word.

app/static/app.js

Ubicación	app/static/app.js
Para qué sirve	Lógica de frontend y orquestación del panel en navegador.
Depende de	index.html y la API expuesta por app/main.py.
Parte del flujo que afecta	Afecta a consulta de datos, render de tarjetas, mapa interactivo, loaders, cachés de UI y acciones del usuario.

Gestiona llamadas a /api/state, /api/articles, /api/incidents-map, /api/image-proxy y el resto de rutas operativas. También implementa skeleton loading, caché en memoria, mezcla de estados, interacción con el mapa y paginación por país. En el archivo hay señales de evolución por hotfix, incluidas definiciones repetidas de algunas funciones, por lo que conviene refactorizarlo cuando el proyecto se estabilice.

app/static/styles.css

Ubicación	app/static/styles.css
Para qué sirve	Estética y layout del panel web.
Depende de	index.html y clases usadas por app.js.
Parte del flujo que afecta	Afecta exclusivamente a la presentación, no a la lógica funcional.

Aplica una estética oscura con acentos verdes, topbar sticky, sidebar con tarjetas, grid de artículos, estilos del mapa y loaders visuales. No introduce lógica de negocio, pero sí determina gran parte de la experiencia de uso.

app/static/cyberad-logo.png

Ubicación	app/static/cyberad-logo.png
Para qué sirve	Activo visual de identidad de CyberRad.
Depende de	index.html y el branding visual.
Parte del flujo que afecta	Afecta a la percepción del producto, no al procesamiento.

Se utiliza en el encabezado de la aplicación y puede reutilizarse en documentación o presentaciones. No es crítico para la ejecución, pero sí para la identidad visual.

app/static/world-outline.svg y world-outline.png

Ubicación	app/static/
Para qué sirve	Base gráfica del mapa mundial.
Depende de	app/incident_map.py, app/brands_map.py y el frontend.
Parte del flujo que afecta	Afecta a la representación visual de incidentes por país.

La aplicación utiliza estas imágenes como superficie base para proyectar puntos. La versión SVG es especialmente útil porque conserva nitidez. Si se reemplazan, conviene revisar la proyección y las dimensiones usadas en los módulos cartográficos.

app/__init__.py

Ubicación	app/__init__.py
Para qué sirve	Marca el directorio app como paquete Python.
Depende de	No tiene dependencia funcional.
Parte del flujo que afecta	Afecta al empaquetado e importación de módulos.

Está vacío, pero sigue siendo importante para que Python trate la carpeta como paquete importable. Es un archivo pequeño con impacto estructural.

data/cyberad_v4.db

Ubicación	data/cyberad_v4.db
Para qué sirve	Base de datos SQLite del sistema.
Depende de	app/db.py.
Parte del flujo que afecta	Afecta a todo: feeds, settings, artículos y feedback.

En el adjunto analizado contiene 51 feeds preconfigurados, 10 settings, 0 artículos y 0 eventos de feedback. Es decir, el paquete se entrega con configuración base pero sin histórico de noticias almacenado. Esto es bueno para distribución limpia, pero obliga a esperar el primer refresco para ver contenido real.

data/.gitkeep

Ubicación	data/.gitkeep
Para qué sirve	Mantener el directorio data en control de versiones.
Depende de	Ninguna.
Parte del flujo que afecta	No interviene en ejecución, solo en la estructura del repositorio.

Es un artefacto habitual para asegurar que la carpeta de persistencia exista incluso cuando la base todavía no se ha creado.

deploy/README_DEPLOY_ES.md

Ubicación	deploy/README_DEPLOY_ES.md
Para qué sirve	Guía de despliegue con Docker y runner autoalojado.
Depende de	Scripts del directorio deploy y Docker Compose.
Parte del flujo que afecta	Afecta al proceso de puesta en producción o automatización CI/CD.

Describe un flujo basado en GitHub Actions self-hosted runner, con instalación de Docker en Ubuntu y runner en Linux Mint. Es útil para operaciones, pero debe leerse junto con el estado real del adjunto: algunas referencias son de entorno recomendado, no de artefactos garantizados dentro del ZIP.

deploy/install_docker_ubuntu.sh

Ubicación	deploy/install_docker_ubuntu.sh
Para qué sirve	Automatiza la instalación de Docker oficial sobre Ubuntu.
Depende de	apt, curl y permisos sudo.
Parte del flujo que afecta	Afecta a preparación del host, no al runtime interno de CyberRad.

Añade el repositorio oficial de Docker, instala engine, CLI, buildx y compose plugin, y mete al usuario en el grupo docker. Es útil para estandarizar hosts nuevos.

deploy/install_runner_mint.sh

Ubicación	deploy/install_runner_mint.sh
Para qué sirve	Automatiza la instalación de un GitHub Actions runner autoalojado.
Depende de	curl, permisos sudo y token válido de GitHub.
Parte del flujo que afecta	Afecta al pipeline de despliegue, no a la aplicación en sí.

Descarga una versión concreta del runner, lo configura con etiquetas y lo instala como servicio. Es útil si CyberRad va a integrarse en un ciclo de build/despliegue interno.

deploy/systemd/cyberad-v4.service

Ubicación	deploy/systemd/cyberad-v4.service
-----------	-----------------------------------

Para qué sirve	Ejemplo de servicio systemd para arranque persistente sin Docker.
Depende de	Entorno Python virtual en /opt/cyberad_v4 y uvicorn.
Parte del flujo que afecta	Afecta a cómo se ejecuta CyberRad como servicio del sistema.

Es una plantilla útil para ejecución nativa. Debe adaptarse al path real del proyecto y al usuario operativo. Tal como viene, es una guía, no un archivo listo para copiar sin cambios.

smoke_test_hotfix23.py y smoke_test_hotfix24.py

Ubicación	raíz del proyecto
Para qué sirve	Pruebas de humo para validar clasificación, prioridades de duplicado, extracción de imagen y mezcla de fuentes.
Depende de	app/rss.py, app/incident_map.py y app/main.py.
Parte del flujo que afecta	Afectan a verificación de calidad antes de desplegar o tocar reglas internas.

En la revisión de este manual ambas pruebas se han ejecutado correctamente. Son especialmente valiosas porque confirman que casos obvios de ruido quedan excluidos y que noticias de interés real pasan como pending o included.

testfeed/feed.xml y testfeed/article-1.html

Ubicación	testfeed/
Para qué sirve	Material mínimo de prueba o simulación.
Depende de	Ingesta RSS y parsing HTML.
Parte del flujo que afecta	Afecta a validación local o depuración manual.

Sirven como base para probar parsing o para crear escenarios reproducibles sin depender siempre de internet. No participan en el panel productivo salvo uso explícito en pruebas.

app/main.py.bak y app/static/app.js.bak

Ubicación	app/ y app/static/
Para qué sirve	Copias de seguridad o histórico manual de cambios.
Depende de	No forman parte del runtime normal.
Parte del flujo que afecta	Afectan al mantenimiento porque pueden generar confusión si se dejan convivir indefinidamente.

No deberían tomarse como fuente de verdad del producto. Conviene archivarlas fuera del árbol principal o moverlas a una carpeta de snapshots para evitar errores humanos durante soporte o refactorización.

6. Funcionamiento interno detallado

6.1 Carga de fuentes

CyberRad mantiene el catálogo de fuentes en SQLite y lo complementa con allowlists y presets de runtime definidos en app/db.py. El proyecto entrega feeds técnicos generales y también búsquedas de Google News orientadas a moda, retail y ecommerce. Durante la inicialización, seed_default_feeds() garantiza que existan feeds base y sync_allowlisted_feeds() alinea el conjunto activo con el perfil seleccionado.

Esto significa que las fuentes no se cargan de forma improvisada cada vez. Hay una política explícita de feeds conocidos, nombres derivados y activación por perfil. En el adjunto analizado, la base lleva 51 feeds preconfigurados.

6.2 Recogida de noticias

La recogida se hace mediante requests HTTP/HTTPS. parse_feed() descarga el feed, detecta si es XML o JSON y extrae las entradas relevantes. Cuando el feed no aporta imagen suficiente, CyberRad intenta enriquecer el artículo abriendo la URL original y leyendo meta tags como og:image o twitter:image. Este comportamiento mejora la interfaz porque evita tarjetas vacías, aunque incrementa las peticiones salientes.

La concurrencia depende del perfil de rendimiento. En low_resource el sistema reduce número de artículos y paralelismo; en full amplía ambos. El backend controla estas peticiones con semáforos async para no disparar todas las fuentes a la vez.

6.3 Filtrado y clasificación

La clasificación tiene dos capas. La primera, en rss.py, es léxica y contextual. Detecta señales ciber, penaliza ruido, interpreta el prompt del operador y emite un score inicial con rationale y tags. La segunda, en intelligence.py, recalcula todo el conjunto con un enfoque híbrido que combina score léxico, score semántico y sesgo de feedback. Los pesos cambian según el relevance_mode: flexible, balanced o strict.

El sistema trabaja con tres estados: included, pending y excluded. Si no hay prompt activo, los artículos quedan como pending por defecto. Si el artículo cae fuera de la ventana temporal seleccionada, se excluye aunque su contenido sea relevante.

6.4 Gestión de imágenes

Las imágenes pueden llegar desde el propio feed, desde HTML embebido o desde la página del artículo. El frontend nunca apunta directamente a la imagen remota cuando puede evitarlo; usa /api/image-proxy para que el backend la descargue, la valide y la optimice. Según el perfil de imagen, Pillow reescala y recomprime el recurso. Esto reduce ancho de banda y mejora la carga del panel, especialmente en low_resource.

La contrapartida es operativa: el backend realiza peticiones adicionales a URLs de imagen. Por tanto, en despliegues abiertos al público conviene controlar quién puede invocar ese endpoint.

6.5 Decisión de ubicación por país

La ubicación por país no se calcula con geolocalización IP ni con servicios externos. Se infiere por señales textuales y reglas internas. Para Marcas, `brands_map.py` busca una relación entre marca afectada, país conocido de esa marca, alias de países y entidades encontradas. Para General, `incident_map.py` busca países explícitos, alias, términos globales, marcas conocidas o pistas en el dominio de la URL. Después proyecta el país al mapa base mediante coordenadas predefinidas.

Este enfoque es suficientemente útil para visualización operativa, pero no equivale a una atribución forense. El mapa debe entenderse como una inferencia de contexto geográfico, no como una prueba absoluta.

6.6 Presentación web y actualización del sistema

El frontend consulta `/api/state` para estado global, `/api/articles` para tarjetas, `/api/incidents-map` para el mapa anual y `/api/incidents-map/country` para el detalle por país. `app.js` aplica loaders, cachés en memoria, mezcla visual de fuentes y renderizado del mapa con zoom y paneo. Además, el backend mantiene un `refresh_loop` opcional que solo actúa cuando `auto_refresh_enabled` está activado.

Al cambiar ajustes importantes como prompt, ventana temporal o modo de relevancia, el backend no se limita a guardar el valor: también agenda un rescoring para mantener coherencia entre configuración y contenido visible.

7. Seguridad del proyecto

CyberRad no es malware ni una herramienta ofensiva. El código adjunto no contiene módulos de explotación, escaneo de puertos, movimiento lateral, persistencia hostil, elevación de privilegios, cifrado malicioso, captura de credenciales ni mecanismos de sabotaje. Su comportamiento principal es recoger feeds, procesar texto y mostrar resultados en una interfaz web.

Técnicamente, el proyecto no genera una vulnerabilidad de red “por sí mismo” por el mero hecho de ejecutarse correctamente en localhost. De hecho, los compose incluidos publican el servicio por 127.0.0.1 por defecto, y el contenedor se ejecuta con usuario no root, sin privilegios extra, con cap_drop ALL y con no-new-privileges. Eso apunta a una intención clara de reducir superficie de ataque.

También es importante lo que el sistema no hace. No abre puertos arbitrarios, no intenta descubrir activos de la red, no escucha protocolos adicionales aparte del servicio HTTP de la propia aplicación, no modifica reglas de firewall, no instala agentes de red y no contacta con C2 ni con servicios de control remoto. La salida de red se utiliza para leer feeds, abrir artículos y descargar imágenes.

Sin embargo, una defensa técnica sería también debe reconocer riesgos normales de despliegue si se configura mal. CyberRad no incorpora autenticación propia en este adjunto. Además, la API permite añadir feeds y existe un endpoint image-proxy que puede forzar descargas HTTP/HTTPS desde el servidor. Si se expone públicamente a usuarios no confiables, eso amplía la superficie y puede permitir un uso abusivo del servidor para hacer peticiones salientes a URLs elegidas por terceros. No convierte al proyecto en ofensivo, pero sí obliga a desplegarlo con criterio.

Qué hace	Lee feeds, clasifica noticias, guarda datos locales, sirve una interfaz web y exporta documentos.
Qué no hace	No explota sistemas, no escanea puertos, no cifra equipos, no roba credenciales, no instala persistencia maliciosa.
Riesgo si se expone mal	Ausencia de autenticación interna y endpoints que desencadenan peticiones salientes a URLs remotas.
Mitigación recomendada	Mantener bind en 127.0.0.1 o proteger con reverse proxy autenticado, VPN o red interna.

7.1 Buenas prácticas básicas de seguridad al instalarlo

- Mantener el servicio accesible solo por localhost cuando sea posible.
- Si se necesita acceso remoto, poner delante un reverse proxy con autenticación y, idealmente, limitar por IP o VPN.
- No ejecutar la aplicación como root en instalación nativa.
- No abrir el puerto a internet sin controlar quién puede invocar /api/feeds y /api/image-proxy.
- Revisar periódicamente dependencias y aplicar actualizaciones con pruebas previas.
- Mantener la carpeta data con permisos apropiados y copias de seguridad.
- Registrar y revisar logs del contenedor o del servicio systemd.

7.2 Riesgos realistas de despliegue y mitigación por escenario

Desde el punto de vista defensivo, el riesgo de CyberRad depende casi por completo del modo de despliegue. Ejecutado en localhost, el impacto de red se limita a peticiones salientes para leer feeds, abrir artículos y descargar imágenes. En ese escenario, el vector más relevante no es una intrusión activa del software, sino un uso incorrecto del host o una salida a internet excesivamente abierta.

Cuando se publica en una red interna o detrás de un proxy, el análisis debe centrarse en dos áreas concretas: la API para alta de feeds y el endpoint de image-proxy. Ambos pueden hacer que el servidor contacte con URLs remotas. Eso no convierte a la aplicación en ofensiva, pero sí significa que conviene tratarla como una aplicación web con capacidad de fetch saliente y aplicar el mismo criterio que se aplicaría a cualquier servicio interno con salida HTTP/HTTPS.

- Escenario 1 - Solo localhost: opción recomendada para laboratorio y uso individual. Riesgo bajo si el host está bien mantenido.
- Escenario 2 - Red interna confiable: aceptable si se protege con reverse proxy, autenticación, control por IP y logs.
- Escenario 3 - Exposición pública en internet: no recomendable en este estado del proyecto sin capa adicional de seguridad, rate limiting y filtrado de salida.

8. Instalación completa desde cero

A continuación se presentan dos caminos: instalación recomendada con Docker y ejecución nativa con entorno virtual Python. Ambos sirven para dejar CyberRad funcional. La vía Docker es la más recomendable para terceros porque simplifica dependencias y deja un despliegue más repetible.

8.1 Instalación recomendada con Docker en Kali o Linux similar

11. Copiar o descomprimir el proyecto en una carpeta de trabajo, por ejemplo ~/CyberRad/cyberrad_hotfix25.
12. Entrar en la carpeta del proyecto.

```
cd ~/CyberRad/cyberrad_hotfix25
export CYBERRAD_UID=$(id -u)
export CYBERRAD_GID=$(id -g)
docker rm -f cyberrad-web 2>/dev/null || true
docker compose -f docker-compose.kali.yml up -d --build
```

13. Esperar a que el contenedor termine de construir y arrancar.
14. Abrir el navegador en <http://127.0.0.1:3000>.
15. Verificar el estado del contenedor y revisar logs si hiciera falta.

```
docker compose -f docker-compose.kali.yml ps
docker logs -f cyberrad-web
curl http://127.0.0.1:3000/api/state
```

Si el objetivo es un despliegue más endurecido o convive con otro servicio en el 3000, usar el compose principal, que por defecto publica en 127.0.0.1:3001.

```
cd ~/CyberRad/cyberrad_hotfix25
export CYBERRAD_UID=$(id -u)
export CYBERRAD_GID=$(id -g)
export CYBERRAD_BIND_IP=127.0.0.1
export CYBERRAD_HOST_PORT=3001
docker compose up -d --build
```

8.2 Instalación nativa con Python

Esta vía es útil para laboratorio, depuración o entornos donde no se quiera usar contenedores. Requiere Python 3.12 y compilación de dependencias tal como aparecen en requirements.txt.

```
cd ~/CyberRad/cyberrad_hotfix25
python3 -m venv .venv
source .venv/bin/activate
python3 -m pip install --upgrade pip
python3 -m pip install -r requirements.txt
python3 -m uvicorn app.main:app --host 127.0.0.1 --port 3000 --app-dir .
```

Aunque el README muestra --host 0.0.0.0 para pruebas rápidas, desde el punto de vista de seguridad es preferible usar 127.0.0.1 salvo que exista una necesidad real de acceso externo y se proteja el servicio adecuadamente.

8.3 Dependencias, estructura y variables relevantes

No se observa un .env obligatorio en este adjunto. La configuración operativa vive principalmente en SQLite y en variables de entorno del compose. Las variables más relevantes son APP_PORT, TZ, CYBERRAD_UID, CYBERRAD_GID, CYBERRAD_BIND_IP y CYBERRAD_HOST_PORT.

La estructura mínima para que arranque correctamente es: raíz del proyecto con app/, data/, requirements.txt y el entrypoint app.main:app. La carpeta data debe ser escribible por el usuario que ejecuta el proceso o el contenedor, porque ahí se guarda la base SQLite y las exportaciones.

8.4 Instalación de Docker en Ubuntu y despliegue con runner

Si el host definitivo es Ubuntu, el propio proyecto incluye un script de preparación de Docker y otro para instalar un GitHub Actions runner autoalojado. Son útiles para automatizar despliegues internos, aunque requieren adaptar URLs, tokens y convenciones del entorno de destino.

```
chmod +x deploy/install_docker_ubuntu.sh
./deploy/install_docker_ubuntu.sh
```

8.5 Cambio de puerto y exposición controlada

El proyecto no obliga a usar un único puerto. En la práctica, el propio paquete contempla 3000 y 3001 según el compose utilizado. Esto es útil cuando el puerto 3000 ya está ocupado o cuando se quiere dejar una convención más segura para el despliegue principal.

En Docker, la forma más limpia de cambiar el puerto es ajustar las variables de entorno del compose principal. En ejecución nativa, basta con arrancar uvicorn con el puerto deseado. Solo debería usarse 0.0.0.0 cuando exista una necesidad real de acceso remoto y se haya decidido una protección adicional delante de la aplicación.

```
# Docker - publicación local en 3001
export CYBERRAD_BIND_IP=127.0.0.1
export CYBERRAD_HOST_PORT=3001
docker compose up -d --build

# Ejecución nativa en 3001
python3 -m uvicorn app.main:app --host 127.0.0.1 --port 3001 --app-dir .

# Exposición a red local solo si es necesario
python3 -m uvicorn app.main:app --host 0.0.0.0 --port 3001 --app-dir .
```

9. Puesta en marcha y uso diario

La primera vez que se abre la aplicación es normal que no aparezcan inmediatamente noticias en el panel. En el arranque, CyberRad lanza un startup refresh en background que descarga feeds, inserta artículos y después recalcula la base. Hasta que ese proceso termine, el panel puede mostrar loaders, estados vacíos o cifras que todavía no reflejan el conjunto final.

9.1 Cómo comprobar que funciona bien

- La URL principal carga y muestra la interfaz sin error 500.
- GET /api/state responde correctamente.
- El bloque de Estado muestra conteos y feeds.
- Tras el primer refresco aparecen artículos en /api/articles.
- El mapa responde en /api/incidents-map y permite seleccionar año.
- Los botones de estado y sector cambian la vista sin romper la sesión.

9.2 Qué significan las secciones y controles principales

Estado	Muestra totales de artículos, incluidas, pendientes, excluidas y feeds activos.
Prompt / filtro interno	Texto usado localmente para orientar el score y excluir ruido; no se envía a un servicio externo según la propia interfaz.
Perfil de rendimiento	Ajusta volumen por feed, concurrencia y calidad de imagen.
Modo de precisión	Flexible, equilibrado o estricto; cambia umbrales y pesos del scoring.
Ventana temporal	Limita noticias por fecha; fuera de esa ventana el artículo termina excluido.
Refresco automático	Activa o desactiva el bucle periódico de refresco.
Ocultar duplicadas	Esconde coberturas repetidas de un mismo evento.
Añadir feed	Permite registrar un feed extra en la base.
Filtros de estado	Todas, Incluidas, Pendientes y Excluidas.
Filtros de sector	General y Marcas.
Mapa anual	Resume incidentes geolocalizados por año, sector, país y tipo.
Exportar Word	Genera un DOCX con noticias del estado solicitado.

9.3 Uso recomendado del operador

Un uso sensato consiste en arrancar CyberRad, esperar a que termine el refresh inicial, revisar primero el mapa y los indicadores, y después usar el prompt interno para estrechar el foco operativo. Las noticias dudosas pueden quedarse en pending; las claramente útiles pasar a included; y el ruido a excluded. Cuanto más consistente sea la reclasificación manual, más útil se vuelve la capa de feedback.

9.4 Qué esperar en el primer arranque

Durante el primer arranque es normal que la interfaz no muestre resultados completos de forma inmediata. La base puede partir vacía, el refresco inicial se ejecuta en background y algunas fuentes tardan más que otras en responder. Por eso, en los primeros minutos pueden verse loaders, contadores parciales o tarjetas todavía sin imagen.

La secuencia esperable es: inicialización de base, siembra de feeds permitidos, descarga de fuentes, inserción de artículos, rescoring y, finalmente, estabilización del panel. En equipos con 8 GB de RAM o perfiles con muchas fuentes, este proceso puede tardar claramente más que en una máquina con 16 GB y perfil balanced.

- Primero debería responder `/api/state` aunque aún no haya artículos visibles.
- Después deberían empezar a aparecer noticias en `/api/articles`.
- El mapa suele poblarse después de que existan artículos ya clasificados y con país inferido.
- Los cambios manuales de estado pueden disparar rescoring adicional y mover temporalmente los conteos.

10. Mantenimiento, validación y solución de problemas

10.1 Errores comunes y posibles causas

No aparecen noticias al abrir	El refresh inicial sigue en progreso; la red no permite salir a feeds; alguna fuente falla; el perfil está limitando volumen; o todavía no se ha completado el rescoring.
El mapa sale vacío	No hay artículos del sector visible; no hay países inferidos; el año seleccionado no tiene datos; o los artículos quedaron excluidos por filtro/ventana temporal.
No cargan imágenes	La URL de origen no es válida, la imagen fue retirada, el content-type no era image/* o el image-proxy no pudo descargarla.
El panel va lento	Demasiadas fuentes, perfil alto para el hardware disponible, recomprensión de imágenes, rescoring amplio o poca RAM.
Los estados cambian raro tras guardar ajustes	Se ha lanzado un rescoring completo y el sistema está recalculando todos los artículos.
No arranca el contenedor	Error de build, dependencia rota, puerto ocupado o permisos inadecuados sobre <code>./data</code> .

10.2 Qué revisar si fallan noticias, imágenes, mapa o filtros

- Ejecutar curl contra `/api/state` y revisar si `startup.in_progress` o `jobs.rescore_in_progress` siguen activos.
- Revisar docker logs `-f cyberrad-web` o la salida del proceso `uvicorn`.
- Confirmar que la carpeta `data` sea escribible y que `cyberrad_v4.db` exista o pueda crearse.
- Verificar conectividad saliente HTTP/HTTPS desde el host.
- Probar con el perfil `low_resource` si el equipo es modesto o tiene 8 GB de RAM.
- Comprobar que el filtro de año del mapa no esté apuntando a un año sin artículos visibles.
- Revisar que el prompt no esté excluyendo por accidente casi todo el contenido.

10.3 Validaciones útiles de funcionamiento

```
# Estado global
curl http://127.0.0.1:3000/api/state
```

```
# Primeras noticias visibles
curl
"http://127.0.0.1:3000/api/articles?limit=5&page=1&status=all&sector=general"

# Mapa general
curl "http://127.0.0.1:3000/api/incidents-map?sector=general&status=all"

# Salud del contenedor
docker compose -f docker-compose.kali.yml ps
```

Como verificación adicional del paquete adjunto, los scripts `smoke_test_hotfix23.py` y `smoke_test_hotfix24.py` se ejecutaron correctamente durante la preparación de este manual.

10.4 Copia de seguridad y restauración

La persistencia operativa de CyberRad está en SQLite. Eso simplifica mucho las copias de seguridad: el fichero más importante es `data/cyberad_v4.db`. Antes de una actualización grande, un cambio de reglas o una limpieza manual de feeds, conviene guardar una copia con fecha.

En contenedor, la protección real depende de preservar correctamente el volumen o la carpeta `data` del host. En ejecución nativa, basta con copiar el archivo de base mientras la aplicación está parada o en una ventana controlada de mantenimiento. Tras restaurar una copia, es recomendable levantar la aplicación y forzar un refresh para verificar coherencia.

```
# Copia rápida de la base
cp data/cyberad_v4.db data/cyberad_v4_backup_$(date +%F).db

# Restauración simple
cp data/cyberad_v4_backup_2026-04-23.db data/cyberad_v4.db

# Verificación posterior
curl http://127.0.0.1:3000/api/state
```

11. Buenas prácticas operativas

- Mantener siempre una copia de seguridad de `data/cyberad_v4.db` antes de tocar reglas, feeds o una actualización grande.
- Probar primero en `low_resource` o `balanced` y solo pasar a `full` si la máquina lo soporta de verdad.
- Actualizar dependencias con cuidado y ejecutar los smoke tests después.
- Añadir feeds gradualmente y observar si aportan valor o solo ruido.
- No mezclar artefactos históricos `.bak` con la fuente de verdad del producto durante una entrega a terceros.
- Establecer una convención de versión única para evitar confusión entre README, ZIP, HTML y nombres de hotfix.
- Evitar exponer la aplicación directamente a internet; usar red interna, VPN o proxy con autenticación.

11.1 Cómo añadir o quitar fuentes correctamente

Las fuentes pueden añadirse desde la propia interfaz o directamente en la base. La vía correcta para uso normal es la interfaz, porque deja el feed registrado y reutilizable. Conviene introducir solo feeds de confianza y observar su comportamiento tras uno o dos refrescos. Si una fuente produce mucho ruido, puede desactivarse o eliminarse. También puede ser necesario revisar si la URL realmente es RSS/Atom/JSON feed y no una página HTML genérica.

11.2 Cómo actualizar sin romper el sistema

16. Hacer copia de seguridad de la base SQLite y de cualquier exportación relevante.
17. Actualizar el código en una rama o carpeta de staging.
18. Levantar el nuevo entorno sin sobrescribir inmediatamente la instancia productiva.
19. Ejecutar `smoke_test_hotfix23.py` y `smoke_test_hotfix24.py`.
20. Comprobar `/api/state`, `/api/articles` y `/api/incidents-map`.
21. Solo después sustituir el servicio anterior o recrear el contenedor definitivo.

12. Conclusión

CyberRad aporta una solución práctica y relativamente compacta para la monitorización de noticias e incidentes de ciberseguridad, con especial atención al filtrado útil y a la representación por sector y país. No intenta ser una plataforma SIEM ni un motor de threat intel corporativo completo; su valor está en transformar múltiples feeds heterogéneos en un panel accionable y configurable con poca infraestructura.

El proyecto puede dejarse operativo de forma fiable si se respetan tres principios: despliegue controlado, mantenimiento disciplinado y criterio al gestionar feeds/configuración. A nivel técnico, el adjunto muestra una base sólida: ejecución local, persistencia simple, endurecimiento razonable del contenedor y clasificación totalmente local. También deja claro dónde están los puntos a vigilar: control de exposición de red, limpieza de artefactos de hotfix y ordenación de la nomenclatura de versiones.

En resumen, CyberRad es un software de agregación, filtrado, priorización y visualización de inteligencia abierta sobre ciberseguridad. Bien desplegado, no supone una amenaza para la red por sí mismo; mal expuesto, hereda riesgos normales de cualquier aplicación web que hace peticiones salientes y no incorpora autenticación interna en este estado del paquete. La diferencia entre ambas situaciones depende del despliegue, no de una intención ofensiva del proyecto.

Anexo A. Resumen de la base de datos

La base SQLite usa cuatro tablas principales: `feeds`, `articles`, `settings` y `feedback_events`. `feeds` almacena nombre, URL y estado de activación; `articles` contiene el histórico operativo completo; `settings` guarda la configuración persistente; y `feedback_events` registra cambios manuales de estado para alimentar el aprendizaje ligero.

feeds	Catálogo de fuentes y su bandera <code>is_enabled</code> .
articles	Noticias normalizadas con URLs, contenido, media, scores, clusters y sector.
settings	Prompt, profile, ventana temporal, auto-refresh, relevancia y opciones de UI/lógica.
feedback_events	Trazabilidad de decisiones manuales del operador.

Anexo B. Rutas API principales

GET /api/state	Estado global, settings, stats, jobs y feeds.
GET /api/articles	Listado paginado de artículos filtrados por status y sector.
GET /api/articles/{id}	Detalle individual de un artículo.
PATCH /api/articles/{id}/status	Cambio manual de estado con posterior rescoring.
POST /api/feeds	Alta de nuevo feed.
PATCH /api/feeds/{id}	Activación o desactivación de un feed.
DELETE /api/feeds/{id}	Eliminación de un feed.
PATCH /api/settings	Actualización de configuración operativa.
POST /api/refresh	Refresco inmediato de feeds.
GET /api/incidents-map	Mapa anual unificado por sector.
GET /api/incidents-map/country	Detalle paginado de incidentes por país.
GET /api/image-proxy	Descarga y optimización de imágenes remotas.
GET /export/docx	Exportación Word de noticias.
GET /export/marcas-map.pdf	Exportación PDF del mapa de marcas.

Anexo C. Resumen rápido de comandos

```
# Kali con Docker
cd ~/CyberRad/cyberrad_hotfix25
export CYBERRAD_UID=$(id -u)
export CYBERRAD_GID=$(id -g)
docker compose -f docker-compose.kali.yml up -d --build

# Logs
docker logs -f cyberrad-web

# Ejecución nativa
python3 -m venv .venv
source .venv/bin/activate
python3 -m pip install -r requirements.txt
python3 -m uvicorn app.main:app --host 127.0.0.1 --port 3000 --app-dir .

# Smoke tests
python smoke_test_hotfix23.py
python smoke_test_hotfix24.py
```

TIMBLAU INDUSTRIES

Fin del documento